

Building a CNN for MNIST Handwritten Digit Classification

✓ Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
# Cell 1: Imports
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```

Explanation of Imports:

- **tensorflow as tf** and **keras**: TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as **tf**

and Keras directly for easy access to their functionalities.

- `from tensorflow.keras import layers`: This imports the `layers` module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- `from tensorflow.keras.datasets import mnist`: This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- `from tensorflow.keras.utils import to_categorical`: This imports the `to_categorical` function, which we will use to perform one-hot encoding of our labels.

✓ Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
# Cell 2: Data Loading and Preprocessing
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add a channel dimension (for grayscale images, it's 1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode the labels
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>:
11490434/11490434 ————— 1s 0us/step

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets (`(x_train, y_train), (x_test, y_test)`).

`x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).

- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.
- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

✓ Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.
3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

```
# Cell 3: Model Definition
# Build the CNN model. Students: Fill in the missing parts!
model = keras.Sequential(
    [
        keras.Input(shape=(28, 28, 1)), # Input layer
        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # Convolutional layer 1
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1
        # Students: Add another Conv2D layer here. Experiment with the number of filters
        # layers.Conv2D(____, kernel_size=(____, ____), activation="____"), # Convolutio
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolutional layer 2
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
    ]
)
```

```

# Students: Add another MaxPooling2D layer here if needed.
# layers.MaxPooling2D(pool_size=(____, ____)), # Max pooling layer 2
layers.Flatten(), # Flatten layer
layers.Dropout(0.5), # Dropout layer
layers.Dense(num_classes, activation="softmax"), # Output layer
]
)

```

Explanation of Layers:

- **keras.Input(shape=(28, 28, 1))**: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- **layers.Conv2D(32, kernel_size=(3, 3), activation="relu")**: This is a 2D Convolutional layer.
 - **32**: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
 - **kernel_size=(3, 3)**: This defines the size of the convolutional filter as 3x3 pixels.
 - **activation="relu"**: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- **layers.MaxPooling2D(pool_size=(2, 2))**: This is a Max Pooling layer.
 - **pool_size=(2, 2)**: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- **layers.Flatten()**: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- **layers.Dropout(0.5)**: This is a Dropout layer.
 - **0.5**: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.
- **layers.Dense(num_classes, activation="softmax")**: This is the output Dense (fully connected) layer.
 - **num_classes**: This is set to 10 because we have 10 classes (digits 0-9).
 - **activation="softmax"**: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

✓ Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
2. **Choose an appropriate loss function and optimizer** for this multi-class classification problem.
3. **In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```
# Cell 4: Model Compilation
# Students: Choose an appropriate loss function and optimizer. Why did you choose these?
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"]) #S
```

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

✓ Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**
3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
# Cell 5: Model Training
# Students: Adjust the batch size and number of epochs. What happens if you change them?
model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1) #Students: F
```

```

Epoch 1/15
422/422 ————— 36s 86ms/step - accuracy: 0.9818 - loss: 0.0574 - val_accuracy: 0.9818
Epoch 2/15
422/422 ————— 36s 86ms/step - accuracy: 0.9845 - loss: 0.0506 - val_accuracy: 0.9845
Epoch 3/15
422/422 ————— 36s 86ms/step - accuracy: 0.9854 - loss: 0.0485 - val_accuracy: 0.9854
Epoch 4/15
422/422 ————— 36s 85ms/step - accuracy: 0.9860 - loss: 0.0431 - val_accuracy: 0.9860
Epoch 5/15
422/422 ————— 36s 85ms/step - accuracy: 0.9877 - loss: 0.0414 - val_accuracy: 0.9877
Epoch 6/15
422/422 ————— 42s 87ms/step - accuracy: 0.9872 - loss: 0.0408 - val_accuracy: 0.9872
Epoch 7/15
422/422 ————— 37s 88ms/step - accuracy: 0.9879 - loss: 0.0388 - val_accuracy: 0.9879
Epoch 8/15
422/422 ————— 36s 85ms/step - accuracy: 0.9885 - loss: 0.0363 - val_accuracy: 0.9885
Epoch 9/15
422/422 ————— 42s 88ms/step - accuracy: 0.9891 - loss: 0.0347 - val_accuracy: 0.9891
Epoch 10/15
422/422 ————— 37s 87ms/step - accuracy: 0.9897 - loss: 0.0314 - val_accuracy: 0.9897
Epoch 11/15
422/422 ————— 37s 88ms/step - accuracy: 0.9895 - loss: 0.0333 - val_accuracy: 0.9895
Epoch 12/15
422/422 ————— 41s 88ms/step - accuracy: 0.9899 - loss: 0.0306 - val_accuracy: 0.9899
Epoch 13/15
422/422 ————— 38s 90ms/step - accuracy: 0.9898 - loss: 0.0298 - val_accuracy: 0.9898
Epoch 14/15
422/422 ————— 36s 86ms/step - accuracy: 0.9908 - loss: 0.0272 - val_accuracy: 0.9908
Epoch 15/15
422/422 ————— 41s 87ms/step - accuracy: 0.9913 - loss: 0.0275 - val_accuracy: 0.9913
<keras.src.callbacks.history.History at 0x7eb902623320>

```

Explanation of Training Parameters:

- **batch_size**: This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **epochs**: One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **validation_split=0.1**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

✓ Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. Run the code cell below.

2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

```
# Cell 6: Model Evaluation
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {loss:.4f}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Test loss: 0.0224
Test accuracy: 0.9922
```

TT B I <> ↺ 🖼️ “ ⋮ ≡ — Ψ 😊 ☰

Section 7: Reflection and Answers to Questions
Conv2D Layer:

The Conv2D layer acts like the “eyes” of the CNN, magnifying glass focusing on specific visual patterns or edges. The `kernel_size` defines how “zoomed in” each filter is. Smaller kernels capture fine details while larger ones detect broader shapes. The number of filters decides how many unique patterns the model can learn at once. More filters often means richer features but also heavier computation.

MaxPooling2D Layer:

MaxPooling is like compressing a high-resolution image into a sharper thumbnail. It keeps only the strongest signals (the maximum values), which reduces computation while making the network more robust to shifts and distortions. This step prevents the CNN from overfitting on irrelevant pixel-level noise and instead forces it to focus on essential features of the digits.

One-Hot Encoding:

One-hot encoding turns each label into a universal “language” for the network. Instead of saying “this is a 3” with a single number, we give the model a vector where the “3 spot” lights up as 1 and everything else is 0. This prevents misleading the network into thinking digits have a magnitude (that 9 > 3). It makes classification cleaner and works better with categorical loss functions.

Flatten Layer:

The Flatten layer is the bridge from vision to logic. After extracting spatial features, Flatten converts that 2D map into a long 1D vector so the Dense layer can treat it like structured data. Turning a map into a list of coordinates, the Dense layer can then classify the information.

Optimizer and Loss Function:

I chose categorical cross-entropy because it directly measures how far the predicted probability distribution is from the target distribution for classification. It punishes confident but wrong predictions. For the optimizer, Adam works well because it adjusts learning rates automatically, combining momentum and adaptive rates to stay stable, and handles noisy gradients well, making training smoother. These choices are standard for CNNs on MNIST.

Section 7: Reflection and Answers to Questions

Conv2D Layer: The Conv2D layer acts like the “eyes” of the CNN. Each filter is like a magnifying glass focusing on specific visual patterns such as strokes, curves, or edges. The `kernel_size` defines how “zoomed in” each filter is. Smaller kernels capture fine details while larger ones detect broader shapes. The number of filters decides how many unique patterns the model can learn at once. More filters often means richer features but also heavier computation.

MaxPooling2D Layer: MaxPooling is like compressing a high-resolution image into a sharper thumbnail. It keeps only the strongest signals (the maximum values), which reduces computation while making the network more robust to shifts and distortions. This step prevents the CNN from overfitting on irrelevant pixel-level noise and instead forces it to focus on essential features of the digits.

One-Hot Encoding: One-hot encoding turns each label into a universal “language” for the network. Instead of saying “this is a 3” with a single number, we give the model a vector where the “3 spot” lights up as 1 and everything else is 0.

Batch Size and Epochs:

I set the batch size to 128 and trained for 15 epochs. This is efficient but still generalizes well. The 15 epochs allowed the model to converge nicely without drifting into overfitting. Smaller batches made training noisier, while too many epochs started to show signs of overfitting. These settings felt like the sweet spot between speed, stability, and accuracy.

Dropout:

Dropout is like sending the network to boot camp, it forces neurons to work harder by randomly silencing 50% of them during training. This prevents "lazy neurons" that over-rely on others, reducing overfitting and boosting generalization. It's basically teaching the network resilience, ensuring it doesn't memorize but actually learns.

Model Architecture:

My CNN has two convolutional layers (32 and 64 filters each) followed by MaxPooling for dimensionality reduction. After feature extraction, the Flatten layer converts the 2D map into a 1D vector so the Dense layer can process it. It's like turning a map into a list of coordinates. The Dense layer can then process and classify the information. I also passes through a Dropout layer (50%) for regularization. The final layer is a Dense layer with softmax activation outputs probabilities for each class. In short: feature extraction → compression → classification.

Performance:

The model achieved a test accuracy of about 98% on the MNIST dataset. I'm satisfied because it shows the architecture is effective on the training data. If I wanted to push higher, I could add more data augmentation (rotations, shifts) to improve robustness. The convolutional block is a great fit for training CNNs on MNIST. But overall, reaching close to 98% accuracy on handwritten digits with such a compact model feels like a sweet spot.

0. This avoids misleading the network into thinking digits have an ordinal relationship (e.g., that $9 > 3$). It makes classification cleaner and better aligned with categorical loss functions.

Flatten Layer: The Flatten layer is the bridge from vision to reasoning. After the CNN extracts spatial features, Flatten converts that 2D feature map into a 1D vector so the Dense layer can treat it like structured knowledge. It's like turning a map into a list of coordinates, the Dense layer can then process and classify the information.

Optimizer and Loss Function: I chose categorical cross-entropy because it directly measures how far the predicted probability distribution is from the true one in multi-class classification. It punishes confident but wrong predictions, which is perfect for digit recognition. For the optimizer, Adam was the clear choice, it adapts learning rates automatically, combining momentum and RMSProp. It's efficient, stable, and handles noisy gradients well, making it a great fit for training CNNs on MNIST.

Batch Size and Epochs: I set the batch size to 128 and trained for 15 epochs. With 128, training is efficient but still generalizes well. The 15 epochs allowed the model to converge nicely without drifting into overfitting. Smaller batches made training noisier, while too many epochs started to show signs of overfitting. These settings felt like the sweet spot between speed, stability, and accuracy.

Dropout: Dropout is like sending the network to boot camp, it forces neurons to work harder by randomly silencing 50% of them during training. This prevents "lazy neurons" that over-rely on others, reducing overfitting and boosting generalization. It's basically teaching the network resilience, ensuring it doesn't memorize but actually learns.

Model Architecture: My CNN has two convolutional layers (32 and 64 filters) with ReLU activation, each followed by MaxPooling for dimensionality reduction. After feature extraction, the Flatten layer converts the 2D maps into a vector, which then passes through a Dropout layer (50%) for regularization. Finally, a Dense layer with softmax activation outputs probabilities for the 10 digit classes. In short: feature extraction → compression → classification. Simple but powerful.

Performance: The model achieved a test accuracy of about 98–99%, which is excellent for MNIST. I'm satisfied because it shows the architecture generalizes well beyond the training data. If I wanted to push higher, I'd experiment with adding data augmentation (rotations, shifts) to improve robustness, or stacking another convolutional block. But overall, reaching close to human level, accuracy on handwritten digits with such a compact model feels impressive.